The Desk Runtime: RLM Desks, Cells, Records and Restarts

Part 2 of the Choir learning series, written to teach as well as to describe. 14 October 2026. Repository: `origin/main` at 980ffaf4e. Evidence is read from code and documents only; nothing in this series touched staging, Node B or a live computer. Where a number comes from a live measurement, it is quoted from the report or problem record that measured it, with the path.

Mutation class of this document: **green** (docs only). The things it describes are mostly **orange** and **red**: the desk runtime, the reducer, wake delivery and restart semantics are protected surfaces.

Evidence labels: **code** (read in the repository at the commit above, with a path and usually a function name), **doc** (an existing Choir document), **upstream** (a vendor or library document), and **inference** (reasoning that has not been tested; treat it as a hypothesis).

How to read this

Section 1 is a primer on the computer-science ideas underneath the desk runtime: agent loops, code as action, actors and mailboxes, delivery guarantees, and crash recovery. If you know actors and idempotency well, skim it. Sections 2 to 7 each take one part of the runtime through the same arc: the general idea, how Choir does it (with file paths), where it is weak (with incidents), and the options. Section 2 is the four desks and their authority. Section 3 is cells and Yaegi. Section 4 is desk worker processes. Section 5 is records, inboxes and wakes. Section 6 is management delegation (S2a and S2b). Section 7 is the restart rules. Section 8 walks one capture through the whole machine. The end has a glossary, five self-check questions, the three files to read first, and sources.

Sibling documents: the tape and Dolt in 01-ledger-and-store.md, the gateway in 03-gateway-and-providers.md, VM lifecycle in 04-computer-lifecycle.md, Autopaper in 05-publication-and-autopaper.md, and the trade-off field guide in 06-reasoning-about-tradeoffs.md.

The short answer

1. **A desk is a persistent agent loop whose actions are Go code.** Each model turn writes a small Go fragment (a *cell*). A Go interpreter, Yaegi, runs it in a separate worker process. The cell calls `choir.*` verbs, but those verbs do not act immediately: they *stage intents* into a tray. When the cell succeeds, the guest daemon's reducer commits the whole tray atomically, together with the desk's inbox cursor (`internal/yaegikernel/choir.go`, `internal/agentcore/rlm_reduce.go`, `commitTray`). A failed cell commits nothing.
2. **Authority is a per-desk verb vocabulary.** `deskModuleSets` in `internal/yaegikernel/choir.go` decides which verbs each desk's cells can even name. Texture alone has `Write`; engineering alone has `WriteFile`, `Exec`, `Freeze` and `Complete`; research alone has the world-reading verbs; management alone has `Verify`, `Delegate` and `ProductAPI`. Doctrine says plainly that this vocabulary is *not* the security boundary (I18); capsules and brokers are.
3. **Desks talk only through addressed records.** A record is a typed commitment (Cast, Ask, Note, Reply, Report, Resolve and so on) that takes the next sequence number in the recipient's inbox, belongs to a conversation and a round, and mints a wake for the recipient (`internal/store/inbox.go`, `planInboxRecords`). The reader's cursor advances in the same transaction as its own acts.
4. **Delivery is at-least-once with idempotent consumers.** The wake path has two durable logs (a Dolt outbox and a SQLite actor log), a projector between them, and a dispatcher that runs one activation per desk at a time. Every hop retries; every hop dedupes on a deterministic key. "The database remembers. The dispatcher delivers" (`internal/actor/actor.go`).

5. **Management used to be one serial turn, and that stopped the paper.** On 14 October one management run held management's only turn for over 30 minutes while captures waited (problem). S2a and S2b (on `origin/main`) let management delegate capture intake to *management tasks*: bounded sub-agents with their own run, session and inbox, K = 4 at once, coordinated by store preconditions rather than locks held across model calls.
6. **A crash restart ends work; a planned restart may resume it once.** The host writes a durable marker before a planned restart; the next boot consumes it before any actor runs (`internal/agentcore/planned_restart.go`). No marker means crash: every pre-boot run, round and work wake is closed "interrupted by a restart", never re-driven. This rule exists because auto-resume after a crash is how crash loops happen, and Choir has had several.
7. **Standing intents survive a crash; their occurrences do not.** An Autopaper source schedule stays active across a crash boot; only the poll in flight closes. A schedule whose poll crashes the computer twice in a row is closed with that cause (`recoverSourceSchedulesAtBoot`, `internal/agentcore/source_schedule.go`).
8. **The runtime's cost is not the interpreter.** On the live paper, Yaegi is about 2% of CPU; the store scans *around* each cell and at each run's end are 24% and 44% (CPU report). The design is sound; its read paths are unindexed.

1. Primer: loops, code as action, actors and crashes

1.1 An agent is a loop

A language model is a function from text to text. An *agent* is a loop around that function: show the model its context, let it choose an action, perform the action, append the result to the context, and repeat until the model says it is done. Everything interesting about an agent runtime is in four questions:

- **What can an action be?** (the action space)
- **Who carries out the action, with what authority?** (the effect boundary)
- **What happens to the loop's state between turns, and across process death?** (durability)
- **How do several loops talk to each other?** (communication)

The rest of this document is Choir's answer to those four questions.

1.2 Tool calls versus code as action

The common action space is the **tool call**: the model emits a JSON object naming a function and its arguments, the runtime runs the function, and the result goes back as text. One model call buys one action. If the model wants to read ten records, filter them, and answer three, that is ten or more round trips, each one re-sending the whole transcript.

Code as action replaces the JSON object with a program fragment. The model writes, say, ten lines of Go that loop over records, filter them and stage three answers. One model call buys a whole small computation. Variables persist between cells (a REPL), so the model can keep a large result in a variable and print only a summary. Three properties follow:

- **Fewer model calls** per unit of work, because loops, filters and joins happen in code, not in conversation.
- **Smaller transcripts**, because bulk data stays in interpreter variables and only views are printed.
- **A different trust problem.** The runtime now executes code it did not write. It needs an interpreter it can kill, a vocabulary of verbs that carry authority, and a separate mechanism (a sandbox) for anything dangerous.

1.3 What "recursive" adds: the RLM

A **recursive language model (RLM)** is a model loop whose code can start further model loops. In Choir a research cell can call `choir.Research` to fan out child research activations, and management can call `choir.Delegate` to start management tasks. Each child is itself a model loop with its own cells. The parent sees the children's results as

values (research) or as later records (management tasks). The design point is that the model sees references and small views while the bulk stays in the environment: "The RLM design point is that the model sees references and small views while the bulk stays in the cell's environment" (LR lever 6).

Recursion needs bounds. Choir allows one level: research children cannot call `choir.Research`, and management tasks cannot delegate (`internal/yaegikernel/choir.go`, the comments on `SessionRoleResearchChild` and `SessionRoleManagementTask`).

1.4 Actors and mailboxes

The **actor model** (Hewitt, 1973; made practical by Erlang) is a way to build concurrent systems without shared mutable state. Its rules:

1. An actor has private state that nobody else touches.
2. Actors communicate only by sending messages to each other's **mailboxes**.
3. An actor processes **one message at a time**. While it handles one, new ones queue.
4. Handling a message may change the actor's state, send messages, and create actors.

Rule 3 is what makes actors easy to reason about: inside one actor there are no races. The cost is **head-of-line blocking**: if one message takes thirty minutes, everything behind it waits thirty minutes. That cost is the whole story of section 6.

In a single process, a mailbox can be a Go channel. In a system that must survive crashes, a mailbox must be durable: a table, with a sequence per mailbox and a **cursor** per reader that says "I have handled everything up to here". The two together give **per-mailbox total order**: every reader sees the same messages in the same order. Order *across* mailboxes is not defined, and a well-designed system does not need it.

1.5 Delivery guarantees and idempotency

Between a sender and a receiver that can each crash, there are three possible promises:

guarantee	how it is built	what can go wrong
at-most-once	send, never retry	a crash loses the message
at-least-once	store, retry until acknowledged	a crash after handling but before the acknowledgement delivers it again
exactly-once	not achievable end to end over an unreliable network	(it is approximated, below)

Real systems build **effectively-once** from at-least-once delivery plus **idempotent handling**: the receiver can see the same message twice and the second time does nothing new. The standard tool is a deterministic **idempotency key**: an identity derived from the content and origin of the message (not a random id minted on each attempt), stored with the effect, so a replay finds the key already present and stops.

The other standard tool is the **transactional outbox**. If a handler must both change its state and send a message, and the state store and the message transport are different systems, a crash between the two writes either loses the message or sends one for a change that never committed. The outbox pattern writes the outgoing message *as a row in the same transaction as the state change*, and a separate relay process reads unsent rows and sends them, at least once. Choir uses exactly this (section 5).

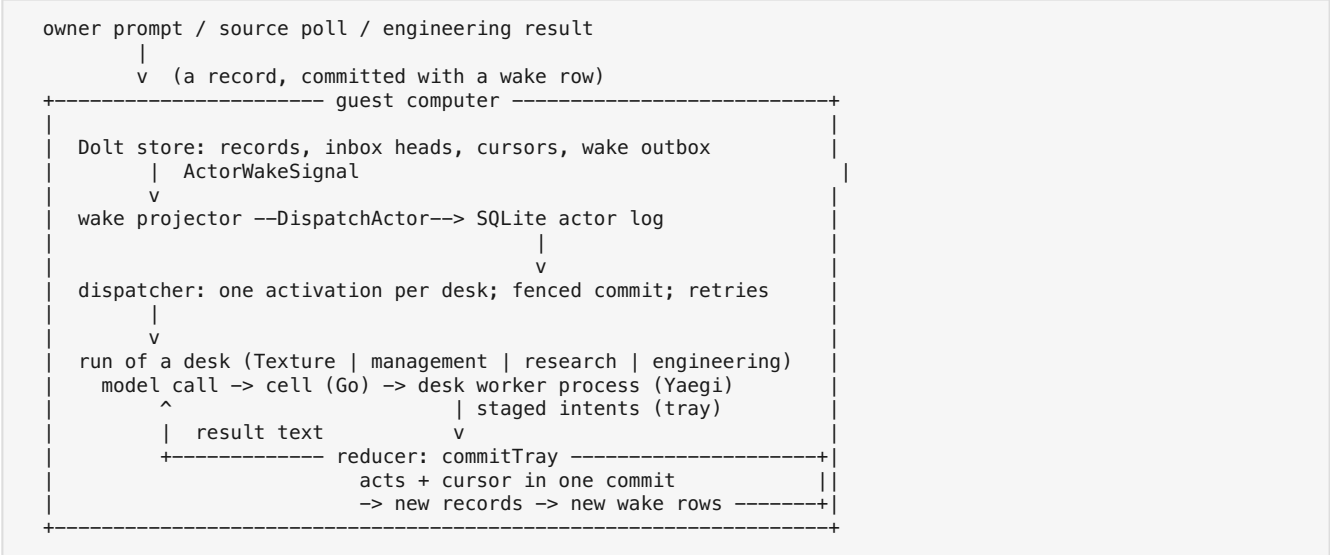
1.6 Crash recovery and crash loops

When a process restarts, what should happen to the work it was doing? There are two answers, and both are used in industry:

- **Resume.** Re-run whatever was in flight. This maximizes progress. It is dangerous when the work itself caused the crash: the restarted process re-runs the same work and crashes again. That is a **crash loop**, and a supervisor that restarts on failure turns it into an outage. Messages that crash their handler are called **poison messages**; queue systems bound retries and divert them to a **dead-letter queue**.
- **Close.** Mark in-flight work as failed with a visible reason and let a live actor (or a person) decide whether to start it again. This loses some progress and never loops.

A middle path distinguishes *why* the process restarted. If the restart was deliberate (an update) the system can resume, because the work did not cause it. If it was a crash, the system closes. To tell the two apart after the fact, the deliberate restart must leave a durable **marker** before it happens. Section 7 is Choir's version.

1.7 The picture



Read it as a cycle: a record wakes a desk, the desk thinks in cells, the cells stage acts, the reducer commits acts as new records, and those records wake other desks.

2. The four desks and their authority

2.1 Primer: least authority and capabilities

The **principle of least authority** says each component should hold only the powers its job needs. There are two ways to enforce it:

- **Access-control lists:** every resource checks the caller's identity against a list. Powers are ambient: anything running as that identity has them all.
- **Capabilities:** a power is an unforgeable reference you hold. If you were not handed the reference, you cannot even name the action. Powers are explicit and can be attenuated (handed on in narrower form).

Choir's desks mix both. The verb vocabulary a cell can import is capability-like: the verb does not exist in a desk's namespace unless the desk's module set admits it. Behind the verbs, brokers and the store check identity and state again. Doctrine I18 states the division of labor: "Yaegi imports present an attenuated capability vocabulary; they do not grant authority. Every consequential operation is independently authorized by current activation capabilities and a trusted broker" (doctrine).

2.2 How Choir does it

Doctrine I26 fixes the ontology: "Each computer has four persistent root RLM desks — management, engineering, research, and Texture — running yaegi Go cells in killable subprocesses." The code makes each authority concrete in three places.

1. The verb vocabulary (`internal/yaegikernel/choir.go`, `deskModuleSets`, and `ChoirExports`, which exports a verb only when the desk's module set admits it):

desk	authority (doctrine)	distinctive verbs (code)
Texture	sole agent writer of canonical document revisions (I1)	`ReadDoc`, `Write`, `Done`, `Wait`, `Reject`, plus `Cast`, `Ask`, `Note`
management	computer-wide coherence; admits engineering by delegated Cast; owns schedules	`Cast`, `Verify`, `Delegate`, `CancelTask`, `CancelAssignment`, `ProductAPI`, `Feed`, `Decline`, `Stories`, `ReadCapture`, the commitment acts
engineering	mutation only inside one assignment's capsule	`WriteFile`, `Exec`, `Freeze`, `Complete`, plus `Report`, `Ask`, `Reply`, `Note`
research	read-only toward the world, full message authority	`SearchStart`, `Searches`, `SearchWait`, `Read`, `ReadCapture`, `SearchWireCorpus`, `PollSource`, `Research`, evidence verbs, the commitment acts
research child	one level of fan-out, no messaging	world-reading verbs only
management task	dispose of leased captures	`Cast`, `Note`, `ReadCapture`, `Stories`, `Feed`, `Decline`, `Done`

Every desk also gets the observation verbs `ReadFile`, `ListDir`, `Context`, `Records`, `Pack`, `Help` and `CellOut`. The comment on research's set states a distinction worth remembering: "read-only world access is not read-only messaging."

2. Who may address whom (`internal/agentprofile/agentprofile.go`, `PolicyFor`): Texture may address research and management; management may address Texture and research; research may address Texture; engineering may address management. The default case returns a policy with no targets, with the comment "This is fail-closed by construction — never add a permissive default here." `Canonical` rejects any profile token outside the live vocabulary with a typed `UnknownProfileError`.

3. The write path itself. Texture's `Write` stages a `TextureTurn`; the reducer commits it through `CommitCellTextureAuthor` (`internal/textureowner/texture_cell_author.go`), which calls `ApplyTextureTurnWithSourceGraph` (`internal/store/texture_turn.go`). That transaction checks the expected head revision, the caller's agent and run, and that the caller's work carries Texture authority, and fails otherwise. The file's own comment: "Texture stays the sole writer of canonical revisions: nothing else here mints one."

Engineering's capsule is the one place where model-written code touches a filesystem and runs processes. An assignment's capsule has a cgroup with `memory.max` of 1 GiB, a CPU quota, 256 pids and a 6-hour fail-closed deadline (`internal/agentcore/engineering_assignment_runtime.go`, `engineeringAssignmentMemoryMax`, `engineeringAssignmentPidsMax`, `engineeringAssignmentDeadline`). Capsules are covered in the isolation review (security review §3.2).

2.3 Where it is weak

- **The vocabulary is enforced by omission, and the handle is broader.** `NewChoirScope` gives every non-research, non-task desk a session handle whose action list includes `ActionExec`, `ActionWriteFile`, `ActionAssign`, `ActionMessage` and `ActionEmit` (code, `internal/yaegikernel/choir.go`, the `actions` slice). For Texture and management, the export layer omits the verbs, so a cell cannot name them; Yaegi's import policy bans `reflect`, `unsafe`, `os`, `net` and friends (`internal/yaegikernel/allowlist.go`, `BannedPackages`), so a cell has no obvious route to the broker except through exported verbs. But the

handle itself would authorize more than the desk needs. *Inference:* if an interpreter bug ever let a cell reach the broker directly, the handle would not stop it. Least authority says the handle should carry only the desk's actions, so the broker check is a real second layer rather than a formality.

- **Management's `ProductAPI` acts as the owner** against an allowlist of product routes (`internal/runtimeprompts/overlays/rlm_management_runtime.yaml`). It is the broadest verb any desk has: it can start and stop source schedules and call computer routes. That is a deliberate ergonomic choice; it is also the verb to watch.
- **Desk workers run with the daemon's identity.** `SpawnDeskSessionWorker` sets a process group, a parent-death signal and a clean environment, but no user or namespace change (code, `internal/yaegikernel/desk_session_worker.go`, `internal/yaegikernel/desk_worker_spawn_linux.go`). The isolation review records the guest runtime as guest root today. So a cell that escaped the interpreter would hold the daemon's full authority inside the guest. Doctrine already says Yaegi is not the boundary; the point is that, for the non-engineering desks, nothing else is either.
- **Old dual paths took a long time to die.** On 11 October most inter-desk traffic still bypassed records: Texture opened research and management through lifecycle controls (77 and 73 uses in the traces), `choir.Message` and `choir.Emit` still existed (problem). Today both verbs return "retired" errors (`choir.go`, `Message`, `Emit`) and the reducer routes every desk's conversation through records (`internal/agentcore/inbox_records.go`, `recordsDesk`). Doctrine I5 ("dual paths are bugs") is the reason that cleanup mattered.

2.4 Options

option	what it buys	cost
Narrow each session handle to its desk's actions	the broker becomes a real second check (defence in depth)	small, red: touches the broker contract
Run desk workers under a non-root uid with no capabilities	an interpreter escape lands as an unprivileged process	medium; workers need read access to the store socket only through the broker
Split `ProductAPI` into named verbs (schedules, computers)	narrower verbs, clearer receipts	widens the verb surface, prompt churn
Keep as is	nothing to build	relies on Yaegi having no escape bugs for non-engineering desks

3. Cells: Yaegi and the in-cell surface

3.1 Primer: interpreters, REPLs and sandboxes

An **interpreter** runs source code without compiling it to a native binary first. Yaegi is a Go interpreter written in Go: it parses Go source, type-checks it, and walks a compiled tree, calling into real Go functions through reflection for the standard library and any symbols the host exports. That makes it about 100 times slower than compiled Go for pure computation (measured below), and it does not matter, because cells are glue.

A **REPL** keeps definitions alive between evaluations, so the next cell can use the variables the last one defined. That is how a desk keeps a large result in memory without printing it.

A **sandbox** is a boundary the code cannot cross whatever it does. An interpreter with a restricted import list is *not* a sandbox: it is a vocabulary. A bug in the interpreter, or a symbol exported by mistake, crosses it. Real sandboxes are made of kernel mechanisms (namespaces, seccomp, cgroups, Landlock) or hardware (VMs). Choir's doctrine is explicit that Yaegi is a vocabulary and capsules are the sandbox (I18, I19).

3.2 How Choir does it

One interpreter per desk session, not per cell. A desk's session is a long-lived worker process holding one Yaegi interpreter (CPU report §2, citing `internal/yaegikernel/session.go`). Each cell arrives as one newline-delimited JSON `SessionFrame` over a Unix socket (`internal/yaegikernel/session_loop.go`). The frame carries the cell's

source and a **cell-start snapshot**: the records in (cursor, head] of the desk's inbox, the Texture document for a Texture cell, the desk's commitment pack, and pending updates. The cell reads them through `choir.Records()`, `choir.ReadDoc()` and `choir.Pack()` with no network round trip.

Verbs stage; the reducer commits. `BindCell` installs a fresh `Tray` for the cell (`choir.go`). A verb like `choir.Cast` appends a `StagedIntent` to the tray and returns in microseconds (`internal/yaegikernel/intent.go`: "delegation, and completion stage into a per-cell in-memory tray in microseconds; the tray ships with the cell result and the guest daemon (autoputer) reduces it after the cell succeeds"). The `SessionResult` returns stdout, any error, the staged intents, and `RecordsShown`, the highest record sequence the cell actually saw, so the commit marks read only what the desk was shown.

This staging design gives each cell **atomicity**: all its acts commit together, or none do. A cell that panics halfway through has staged three of six acts, and none of them reach the world. The reducer's comment: "Failed, timed-out, or poisoned cells reduce nothing and advance no cursor" (`internal/agentcore/rlm_reduce.go`).

Refusals are never lost. Every exported verb whose last result is an `error` is wrapped by `recordRefusals`, so a refused verb is listed in the cell's result even when the model wrote the call as a bare statement and ignored the error. That fix came from an incident where a verb error "vanishes as final expression" (problem).

Imports. Model cells may import a small safe stdlib set (`fmt`, `strings`, `bytes`, `time`, `math`, `sort`, `strconv`, `encoding/json`, `regexp` and a few others), and never `unsafe`, `reflect`, `runtime`, `plugin`, `syscall`, `os`, `os/exec`, `net`, `net/http` or `io/ioutil` (`internal/yaegikernel/allowlist.go`, `DefaultSafeStdlibPackages`, `BannedPackages`).

Failure classes. Yaegi errors carry a reuse disposition (`internal/yaegikernel/eval.go`): an import or compile error is `preserve` (nothing ran; the session stays usable), while a runtime error, panic or timeout is `unsafe_to_reuse` and poisons the session (`session.go`: "A Session is poisoned by any failure that may have executed"). The reasoning: once code has run partway, the interpreter's heap may hold half-updated state, so the only safe recovery is a fresh process.

Output bounds. A cell's printed output is capped at 2 MiB in the worker (`eval.go`, `maxEvalOutputBytes`). LR lever 6 added `choir.CellOut` (`internal/yaegikernel/cell_output.go`): a large print goes to the transcript as a head and tail view with a handle, and the whole text stays in the session for a later cell to read. The measurement that motivated it: research cells' p90 stdout was 27,042 characters, and one cell printing six search records raised the next call's input from 7,211 to 30,221 tokens until compaction (LR lever 6).

Deadlines. A management cell that sets no `timeout_ms` gets a host deadline of 300 seconds (`internal/agentcore/tools_desk.go`, `managementCellDeadline`). The constant's comment cites the incident: run 305a582d's cell 6 ran 860,565 ms before its worker died. Other desks have no host deadline, because research legitimately waits on `choir.SearchWait` (`deskCellHostDeadline`).

3.3 Where it is weak

- **The cost is around the cell, not in it.** Measured on the paper at 16:46 UTC on 14 October: Yaegi workers and the rest of the guest's small processes together are about 2% of CPU; the host-side work around each cell is 24%, and of that, building the commitment pack (`actingCommitmentPackForDesk`) is 11% and loading the cell's records (`loadCellRecords` → `ListInboxRecords`) is 9%, both by unindexed scans that parse every row's JSON (CPU report, "The short answer" and §1b). You can see the scan in `ListInboxRecords` (`internal/store/inbox.go`): it lists every commitment record for the inbox by metadata, then filters by sequence in Go. 01-ledger-and-store.md explains why that is slow in Dolt and what an index would change.
- **Every cell is parsed and compiled from scratch.** Six to twelve `go/parser` passes for import checks and normalization, then a Yaegi compile; compiled programs are not cached. A glue cell costs 1.28 ms under Yaegi against 9.6 µs native, and allocation doubles as a session ages past 500 cells (CPU report §2). This is small in absolute terms.

- **The transcript grows with every call.** In run 305a582d, input grew from 13,668 to 137,087 tokens per call over 58 calls: 4.1 million input tokens for one management turn (delegation design §2.4). Lever 6 bounds one source of growth (prints), not the turn's length.
- **A runaway cell is a whole-worker failure.** The 860-second cell ended with `frame: read header: EOF`; the worker's exit cause was not recorded. The hypothesis (H1 in the design note) is a hand-written index loop that never advanced. A host deadline now bounds management; other desks rely on the model choosing a timeout.

3.4 Options

The CPU report's table (§2) is the right summary, and its verdict is clear: keep Yaegi.

option	gain	cost	verdict (CPU report)
keep Yaegi	—	—	keep: it is not where the cycles go
cache the import check and normalization parse	~0.1 ms per cell	small, yellow	only with other work in the same files
native helpers for hot glue (filters, JSON shaping)	ms to µs per cell	widens the verb surface	only for helpers a profile shows
cap session age (respawn after N cells)	bounds the +19% aging	a respawn costs ~0.15 CPU-s	after a profile shows aged sessions
prefork pool or a small worker binary without Dolt	spawn ~0.15 → ~0.01 CPU-s; ~40-60 MiB less RSS per worker	binary must track the verb surface	worth it for RAM
a different language (Starlark, JS, Lua, wasm)	none that matters	rewrites prompts, cell ABI and evidence	no

The lesson generalizes: **profile before you replace a component.** The interpreter looked like the obvious suspect; the profile pointed at store reads.

4. Desk workers: processes, sessions and leaks

4.1 Primer: why a separate process

Running model-written code in the daemon's own address space would make every interpreter bug a daemon bug: a runaway allocation would take down the guest's main process, and a panic would need careful recovery. A separate process gives four things for free:

- **A kill switch.** `SIGKILL` on a process group ends the worker and everything it started, without cooperation.
- **A memory ceiling.** `RLIMIT_AS` caps a process's virtual address space; an allocation past it fails inside the worker, not the daemon.
- **Crash containment.** A worker crash is a closed socket for the daemon, which it can turn into a cell failure.
- **Lifetime binding.** On Linux, `Pdeathsig` asks the kernel to kill the child if its parent dies, so a daemon crash cannot leave orphaned workers.

The cost is start-up time and memory per process. That creates the classic trade-off of **pooling**: keep processes around to amortize start-up, and accept that a pool needs an eviction policy, or it leaks.

4.2 How Choir does it

`SpawnDeskSessionWorker` (`internal/yaegikernel/desk_session_worker.go`) re-executes the autoputer binary in `desk-session` mode with:

- a Unix socket pair, the child end passed as file descriptor 3, so "worker traffic never touches process stdio";
- `Setpgid: true, Pdeathsig: SIGKILL` (`desk_worker_spawn_linux.go`, `workerSysProcAttr`);
- an environment of only `PATH`, `TMPDIR` and the session configuration;

- a 10-second ready handshake: "a process that starts but never proves readiness ... is killed, never trusted" (`sessionReadyTimeout`);
- an 8 GiB `RLIMIT_AS` applied inside the worker (`internal/agentcore/tools_desk.go`, `deskWorkerMemoryLimitBytes`; `desk_worker_memlimit_linux.go`, `ApplyWorkerMemoryLimit`). The comment explains the size: the re-executed binary's linked address space needs headroom, so "the cap binds growth, not startup size".

The pool is `deskSessionWorkers` in `tools_desk.go`:

- **Keyed by desk agent** (`deskSessionKey`): a desk run uses `agent:<id>`, so the next run of the same desk continues in the same REPL ("A desk's session outlives its run (PD2)"). A research child gets a session of its own, because "a child that joined its parent's session would wait on the parent's cell forever".
- **One REPL per desk.** `acquire` waits, outside the pool lock and bounded by the caller's context, while another run's cell is busy in the session, and tells the model when it shared the session.
- **Eviction.** Idle sessions close after 30 minutes (`deskSessionRetention`), and at most 12 idle sessions are kept, least recently used first (`deskSessionMaxIdle`, `reclaimLocked`).
- **Death and drift.** A dead worker, or one whose configuration differs from the requested one, is closed and replaced on the next acquire.

Measured on the paper: 12 worker processes at 15:25 UTC on 14 October, 17 to 127 MiB anonymous memory each, against 2.49 GiB for the main process (CPU report §2). The memory-return work set the worker's Go soft memory limit to 1/16 of guest RAM, at least 256 MiB, and notes that "workers are not measured yet" (memory return).

4.3 Where it is weak

- **The leak that motivated the pool.** Before PD2, the pool was keyed by run, and a run that ended normally left its worker alive: "Every Texture turn, every research run and every management wake adds one idle process ... Nothing reclaims it until the computer restarts" (problem). The fix is the agent key plus retention and LRU bounds above. The lesson: **every cache needs an eviction rule that runs even when nothing else happens.**
- **Which brings up the current gap.** `reclaimLocked` runs only inside `acquire` and `done` (code: those are its only callers in `tools_desk.go`). *Inference:* on a computer that goes quiet, up to 12 idle workers (and their tens of MiB each) stay alive until the next cell runs anywhere, however long that is. That matters exactly when the platform wants to reclaim memory from idle computers (04-computer-lifecycle.md).
- **The worker is the whole autoputer binary.** Each worker links the Dolt engine and gRPC it never uses; the CPU report estimates 40 to 60 MiB less RSS per worker with a small dedicated binary.
- **The 8 GiB cap is on address space, not resident memory.** It stops a runaway allocation, but twelve workers each well under the cap can still together press a 4 GiB guest. The Go soft limit is the real bound, and it is a reasoned default, not a measurement.
- **Exit causes are not recorded.** The worker's stderr is a buffer read on spawn failure; a mid-cell death reaches the model as an EOF on the frame header (section 3.3).

4.4 Options

option	buys	cost
a reaper timer that runs `reclaimLocked` every few minutes	idle computers return worker memory	tiny, orange
record exit status and signal on worker death	turns EOFs into causes (OOM, limit, panic)	small
a dedicated `desk-worker` binary	~40-60 MiB per worker, faster spawn	must stay in lockstep with the verb surface
per-worker cgroup leaf with `memory.max`	a real resident-memory bound	needs cgroup delegation in the guest

5. Records, inboxes and wakes

This is the heart of the runtime. It is also where most of the historical bugs lived, so it is worth reading slowly.

5.1 Primer: from messages to records

A plain message says "here is some text for you". A **commitment record** says "I, this desk, at this time, cast this objective to you, in this conversation, round 3; here are my hypotheses and the evidence I cite." Records are typed, immutable, addressed and numbered. A reply is a new record that refers to the old one, never an edit. That makes the set of records an **audit log** of who asked whom for what and what came back: the *commitment ledger* that Texture renders for the owner (01-ledger-and-store.md covers the tape and the ledger's storage).

Three ideas organize records:

- **Inbox:** a durable mailbox named after the recipient desk agent: `texture:<doc>`, `research:<doc>`, `management:<owner>`, `engineering:<assignment>`, and since S2a `management-task:<id>`.
- **Conversation:** the pair of inboxes talking, for example (`management:<owner>`, `texture:<doc>`).
- **Round:** one request and its answer inside a conversation. A Cast opens a round; a final Report from the recipient, or a system closure, closes it. Rounds make "who owes whom an answer" a derived fact rather than something a model must remember.

5.2 How a record is committed

`planInboxRecords` (`internal/store/inbox.go`) does, for each record in one batch:

1. Validate its shape, refuse a duplicate id in the batch, and refuse a record whose id is already committed (`CommitmentRecordExists`).
2. Load the inbox head and take the next sequence: `head.value.Seq++`. The head is advanced by compare-and-set, "like the arrival ordinal".
3. Derive the conversation id from the inbox pair, load the conversation head, and step it (`stepConversation`): open a round, close one, or refuse an out-of-turn act. For example, only the caster of an open round may Cast into it, and only the recipient closes it.
4. Write the record as a `choir.commitment_record` object, with `inbox`, `inbox_seq`, `conversation_id` and `round` in its metadata.
5. Mint one **wake row** for the recipient, unless the record is a system record that wakes no one (`inboxRecordWakes`; `RecordWakesReader` in `internal/store/management_tasks.go` is the single predicate since S2a).

The file's header explains a subtle design choice: the **reader cursor** is its own object, not a field on the agent record, because "the agent record is rewritten under CAS by most lifecycle transitions, so a cursor there would make every advance race unrelated agent writes". A dedicated object "commits in the same PutBatchConditional as the reader's effects, which is the atomicity D3 asks for." In plain terms: **a desk's acts and its "I have read up to here" commit together, or not at all.** That is the property that makes at-least-once delivery safe: if the cell's commit fails, the cursor did not move, and the records will be shown again.

5.3 How a cell's tray is reduced

`commitTray` in `internal/agentcore/rlm_reduce.go` is the reducer. It:

- folds Texture's Cast, Ask and Note into the same cell's Texture turn, so "a dispatch commits with the turn" (protocol P2a, `foldTextureDispatch`);
- routes research, management and engineering acts through `commitRecordsRoute`, so "their acts to Texture and their cursor advance commit as one transaction";
- commits the remaining intents by kind: `Complete` and `Freeze` on a detached context with a 3-minute bound ("freezing must outlive cell teardown"), `Verify`, Texture turns, and semantic acts;
- finally advances the inbox cursor to the cell's high-water mark and marks the receipt committed.

Two more pieces make the reduction idempotent:

- **Deterministic identities.** `intentIdempotencyKey` derives an envelope's key from the cell id, the intent's local id, and a hash of its destination and content. The record id of a commitment is likewise derived from the cell ("a re-reduced cell re-derives the same id and the not-exists condition re-mints nothing", `commitActIntent`). The `internal/workitem` package does the same for work items: `ObjectiveFingerprint` hashes owner, trajectory, parent run and the normalized objective.
- **A named recovery path for a cross-store commit.** `recoverPartialActCommit` exists because "the object graph, channel log, and run-memory cursor do not expose one shared transaction". If a record exists but the cursor or the envelope is missing, the whole tray is replayed, and the deterministic keys make the replay converge "without double-minting a record or duplicating mail".

This is a textbook pattern worth naming: **when you cannot have one transaction, make every step idempotent and make replay the recovery.**

5.4 How a wake becomes a turn

The delivery path has more hops than it looks. In order:

```
commit (Dolt): record + wake row (choir.actor_wake_outbox)
    |  store.signalActorWakesIfAny -> ActorWakeSignal (cap 1)
    v
runActorWakeProjector (guest goroutine)
    sweepActorWakeOutbox: ListUnprojectedActorWakes
      - crash-boot gate: preBootWorkWake -> MarkActorWakeInterrupted
      - pacing: <= 64 dispatches per sweep, <= 4 per target desk
      - DispatchActor ... then MarkActorWakeProjected
      - on failure: backoff 1 s, 2 s, 4 s ...; after 5: exhausted
    v
actor log (SQLite, actor_updates; Append is idempotent on UpdateID)
    v
Dispatcher: PendingAgents -> one serial activation per actor
      - fenced commit of {emitted updates + state head} at an epoch
      - MaxAttempts 8, MaxDeferrals 64, then a dead-letter sink
    v
actor handler -> WakeRecordsInbox(deskAgentID)
      - management: only the persistent management agent
      - an active run of this desk?  -> do nothing (it will see it)
      - unread records?               -> StartRunWithMetadata
```

Sources: `internal/store/actor_wake_signal.go`, `internal/agentcore/actor_wake_projector.go`, `internal/agentcore/runtime.go` (`sweepActorWakeOutbox`), `internal/actor/actor.go`, `internal/actor/dispatcher.go`, `internal/actor/log_sqlite.go`, `internal/actorruntime/adapter.go` (the SQLite log, `MaxAttempts: 8`, `MaxDeferrals: 64`, `ErrorSink`), and `internal/agentcore/inbox_records.go` (`WakeRecordsInbox`).

Several design decisions are visible here:

- **Signal, then sweep; never trust the signal alone.** The store signals only after a batch with wake rows has committed, so "a sweep can never run ahead of the write it was told about". The projector also sweeps on a one-minute audit timer that "exists only to expose a missed signal", and logs when the audit finds work a signal should have announced.
- **Serial per actor.** The dispatcher "never runs two activations of the same actor; events arriving during an activation are folded by the next." That is actor rule 3 from section 1.4.
- **Fencing.** Each activation commits at the epoch it started from; "a stale or preempted activation's commit fails its epoch check", and its emitted updates die with it. A **fencing token** is the standard defense against a slow process that thinks it is still in charge.
- **Bounded retries with a visible end.** Projector dispatch failures are retried five times with doubling backoff, then marked exhausted, "a recorded, visible fate" (O1). The dispatcher routes a wake that fails eight times to a dead-letter mailbox. Both are the poison-message defence from section 1.6.
- **Pacing.** The 64-per-sweep and 4-per-desk limits came from an incident (below): "A boot storm that mints thousands of wakes for the same desk collapses to a bounded burst per pass."

5.5 What happens when a turn ends

When a records desk's run ends, `recheckRecordsInbox` (`inbox_records.go`) does three things:

1. **Marks read** only the records the turn was shown (`markTurnRecordsRead`, using `RecordsShown`). A record that arrived during the turn and was never shown stays unread.
2. **Closes the rounds the run left open** (`closeRoundsOfEndedRun`): if a research or management run ends without a final Report, each round it owed gets a system final Report with reason `run_<state>`, and the caster is woken, because "this closure is news the caster must act on". Management keeps a Texture's round open while an engineering assignment it cast is unfinished (`childAssignmentsHoldRound`).
3. **Re-arms the desk** if a record that wakes its reader was committed after this turn began, by dispatching an `inbox_record` wake. Without this, a record that arrived mid-turn would wake nobody (the desk was busy when it arrived) and wait for the next unrelated event.

Step 3 is the general shape of a **lost wake-up** bug and its fix: any system that suppresses wakes while busy must re-check when it becomes idle.

5.6 Where it is weak

- **The run-end check is the single most expensive thing on the paper.** `closeRoundsOfEndedRun` → `OpenRounds` → `ConversationState` is 44% of the paper's guest CPU: at the end of every desk run it scans every commitment record matching the inbox, then once per conversation, with `JSON_EXTRACT(CAST(metadata AS JSON))` on every row (CPU report; problem). The conversation heads are already maintained as cached objects in `planInboxRecords`; the fix the report proposes (levers C1 to C3) is to index the metadata fields and fold rounds incrementally. The design is right; the read path ignores the caches it already writes.
- **Two durable logs for one delivery.** The Dolt outbox and the SQLite actor log are separate stores with separate failure modes. The SQLite log uses one connection with a 60-second busy timeout (`actorruntime/adapter.go`). The dispatcher's own comment notes that "multi-dispatcher failover [is] unresolved, and a single dispatcher per computer is assumed." *Inference:* the projector's relay is the transactional-outbox pattern done correctly, but every hop is a place a wake can be delayed, and debugging a missing turn means reading both stores.
- **The re-arm storm.** On 6 October the owner's computer rebooted repeatedly under host memory pressure, and every boot re-minted about 2,100 wakes (`minted 2106 pending wakes` and similar, eight boots in eight hours) from an O(all objects) migration scan; the projector then spent hours dispatching dead wakes while owner work queued behind them (problem). The per-sweep pacing and the dead-wake disposal (`ErrNoPendingActorOccurrence` → mark projected) are its repairs. The lesson: **a recovery path that re-arms everything on boot is a load generator.**
- **The retry tracker is process-local.** `wakeRetryTracker` is in memory "by design: after a restart, pre-boot work follows the restart rule, and a wake that fails again starts a fresh, still bounded, budget." That is a reasoned choice. It means "bounded" is per boot, not per wake's lifetime.
- **A desk turn is one turn per desk.** `WakeRecordsInbox` starts a run only when no run of that desk is active. This is the actor model's serialism, applied at the granularity of a whole model turn, which can last many minutes. Section 6 is the consequence.

5.7 Options

option	buys	cost
index `inbox`, `sender_inbox`, `conversation_id`; read conversation heads instead of folding	the 44% and 9% scans become point reads	small, orange (query changes)
an incremental round fold at commit	run-end check becomes O(rounds touched)	medium
one durable log for delivery (fold the actor log into Dolt, or the outbox into SQLite)	one place to look, one failure mode	large, red; the current split works

option	buys	cost
record retry counts durably	"bounded" across boots	small; conflicts with "a crash ends work" only if misused

6. Management delegation: S2a and S2b

6.1 Primer: queues, head-of-line blocking and leases

Head-of-line blocking happens when a single serial worker processes a queue and one slow item delays every item behind it. The fixes are always some mix of: make items smaller, add workers, or separate slow item types from fast ones.

Little's law relates the three quantities of any stable queue: L = λW (items in the system = arrival rate × time each spends in it). If captures arrive at 0.45 a minute and each takes about 2 minutes to handle, about one is in service on average; if they arrive at 4.5 a minute, about nine must be in service at once to keep up.

Adding workers creates a new problem: two workers must not handle the same item, and their effects must not conflict. Two families of tools:

- **Locks** (pessimistic): hold exclusive access while you work. Bad when the work includes a minutes-long model call.
- **Optimistic concurrency** (compare-and-set): work without a lock, then commit only if what you read has not changed; otherwise retry with fresh data. Good when conflicts are rare and work is slow.

A **lease** is a time- or state-bounded claim that gives one worker the sole right to act on an item. Leases are how work queues hand items to workers without holding locks.

6.2 The problem

Management was one run at a time over one cursor (`WakeRecordsInbox`), and every kind of work went through that run: capture dispositions, a Texture's Casts and Asks, support judgement, engineering admission. The design note counted 17 hours of the live paper's management inbox: 154 runs; management was in a run 46% of the window; 105 of 132 capture arrivals found management already busy; duration p95 812 s and max 3,601 s; arrival-to-handling p95 about 21 to 25 minutes (delegation design §2.2-2.3).

On 14 October at 03:50 UTC the owner noticed the newest article was 28 minutes old. One management run (1565c043, model `longcat-2.5-preview-free`) had held the only turn since 03:13 while it checked citations itself, one capture per cell, with model calls of 9 minutes and input growing to 113,382 tokens (problem). Nothing was lost; everything waited.

The note names the three costs of a serial turn: **latency** (everything waits for the slowest thing), **fate-sharing** (one stuck cell stops all intake), and **tokens** (one transcript grows over 58 calls).

6.3 How Choir does it now (S2a)

S2a, on `origin/main` (commits a2d3d1f81, 1e218156e and neighbours), gives management a verb, `choir.Delegate("intake", objective, refs, grouping)` (`internal/yaegikernel/choir.go`). It hands captures records to **management tasks** and "returns at once: nothing waits on a task."

A management task (`internal/agentcore/management_tasks.go`, header):

- **is a run** of profile `management` and role `management_task`, reading its own inbox `management-task:<id>` in its own desk session;
- **sends outward as** `management:<owner>`, with the task named in provenance, so a Texture still has one conversation with management;

- **has a narrow verb set**: `Cast`, `Note`, `ReadCapture`, `Stories`, `Feed`, `Decline`, `Done` (`deskModuleSets[SessionRoleManagementTask]`); it cannot cast engineering, verify, call the product API, or delegate;
- **is bounded**: at most 12 captures (`MaxTaskCaptures`), a 10-minute run deadline (`managementTaskDeadline`), a 3-minute bound per model call (`managementTaskCallTimeout`);
- **has a deterministic id** derived from the delegating cell (`managementTaskID`), "so a replayed cell opens the same task (and the store refuses a second one)".

The coordination is done by the store, not by locks:

- **The lease is the open round.** "There is no lease table." A capture is leased to a task exactly while the task's round naming it is open (design §4.3). Closing the round (final Report, crash closure, deadline, cancel) frees the lease with no extra write. That is standing question 5 in action: one authority per state.
- **A disposition index.** A computer-scoped object keyed by capture id, written in the same batch as the disposition record and conditioned on being absent, so a capture cannot be disposed of twice (design §6.4).
- **The story-library head.** Two tasks might open two stories for the same event. The store keeps a count of story opens; the host stamps each `texture:new` Cast with the head the run last saw through `choir.Stories()`; a Cast stamped behind the current head is refused, naming the stories opened since (design §6.1). That is optimistic concurrency: no lock across the model call, a cheap check at commit.
- **The host mints the final Report.** The model ends a task with `choir.Done(note)`; the host computes the residuals (leased captures minus durable dispositions) and mints the Report. A clean Report (no residuals, no error) does not wake the parent; one with residuals does (`RecordWakesReader`). The model does not get to claim it was clean.

6.4 S2b: K at once

S2a ran one task at a time, and at 28 sources it fell behind: 149 of 154 captures undisposed after 34 minutes, queue wait p50 425 s (problem). S2b (commit 433ce4d6e on `origin/main`) runs K tasks at once:

- **K = 4 by default, at most 8** (`provideriface.NormalizeManagementTaskSlots`, `RUNTIME_MANAGEMENT_TASK_SLOTS`).
- **A durable slot starter.** `startQueuedManagementTasks` holds a mutex, counts running task rows (a claimed row with no run yet counts as busy), and starts queued tasks while slots are free. A queued task is "an open task round with no run", not an in-memory queue; its start is an outbox wake of a restart work kind, so a crash boot interrupts it and a planned boot delivers it once.
- **Shared model-call pacing.** Task model calls join the research children's computer-wide spacer: 1.5 s apart, 40 a minute (`internal/agentcore/research_children.go`, `researchChildCallInterval`; renamed `subRLMCallSpacer` in S2b), under the gateway's 60 a minute per computer.
- **A live rollback.** The owner setting `management.serial_tasks` forces K = 1 on the next start, with no deploy. That matters because "the paper is always busy, and deploys never restart a busy computer" (design §18.1, FM31).

Note that the design note's §17 and §18 headings say "not pushed"; the commits are on `origin/main` now (code: `git log origin/main` shows 433ce4d6e and a2d3d1f81). Whether S2b is deployed on the paper was not checked for this document.

6.5 Where it is weak

- **Answer tasks are not open.** `Delegate` refuses `kind == "answer"`: "answer tasks are not open yet; answer the Texture yourself". A Texture's Asks still wait on the parent's turn.
- **Engineering delegation has its own history.** On 10 October a management Cast to engineering died because the caster's run had already completed when the deferred spawn reached bind, and later casts were refused because earlier work items were never closed; the model then told Texture, wrongly, that work

was "already covered" (problem). The lesson recurs: **authority that is tied to a short-lived run dies with the run.** S2a's design avoids it by tying the task to its round, not to the parent run (doctrine I3: "parent/child is not a control ontology").

- **The store is a single writer.** Every commit serializes on the object graph's engine lock (`internal/objectgraph/engine_mutex.go`). K = 4 tasks commit milliseconds at a time, so the bet (hypothesis H3) is that waits stay small; S2b exposes the lock's waits in `/api/runtime/observability` to measure it rather than assume it.
- **Every task is a full model loop.** The design estimates 4 to 6 calls per intake task. The owner's ruling O3 points further: high-confidence Jev labels that name an existing story become system Feeds with no model call (design, "Owner rulings"). That is the deepest fix for a queue: remove items from it.

6.6 Options, as the panel weighed them

shape	how it scales	why (not)
one serial turn (before)	does not	head-of-line blocking; the 14 October outage
parallel management turns over inbox partitions	some	rejected by all six panelists: "a single cursor and a single active run ... would need the same lease machinery anyway" (adjudication)
management tasks, K at once (S2a, S2b)	K × task rate	chosen: bounded, leased, coordinated by store preconditions
deterministic intake (Jev-labelled feeds as system acts)	no model call per item	the destination after tasks; recorded acts that can be overridden

7. Restarts: crash versus planned

7.1 Primer: what a restart must decide

Return to section 1.6. After a restart, a system must decide for each piece of in-flight work: resume or close. The risk is asymmetric. Closing work that could have resumed costs some progress and a visible failure. Resuming work that caused the crash costs a crash loop, which can take the computer and its owner's attention for hours. The design principle Choir adopted is in `CLAUDE.md`: "A crash restart never resumes work ... Auto-resume after a crash is how crash-restart loops happen."

The engineering trick is to make "was this restart planned?" a **durable fact written before the restart**, not a guess made after it.

7.2 How Choir does it

The marker (`internal/agentcore/planned_restart.go`):

- `WritePlannedRestartMarker` writes `<store>.planned-restart` with reason, target, time and work-state epoch, via the crash-safe file idiom: write a temp file, `fsync` it, rename it into place, `fsync` the directory. (Without the directory sync, a rename can be lost on power failure even after the file's own data is durable.)
- The host asks for it through `HandleInternalPlannedRestart` (`POST /internal/runtime/planned-restart`). A busy guest answers 409 `busy` and writes nothing, "so the host leaves it for its next idle moment". Busy means running runs or self-development operations mid-transaction (`plannedRestartBusy`), and an unreadable operation list counts as busy: fail closed.
- `ConsumePlannedRestartMarker` reads and **removes** the marker before any work resumes. It is planned only if the marker is readable, younger than 15 minutes (`plannedRestartMaxAge`), and carries this binary's `CurrentWorkStateEpoch` (2 today). A marker that cannot be removed is treated as a crash, "rather than risk a resume loop". The comment states the invariant: "a resume that crashes the guest leads to a markerless (crash) boot: one resume per planned restart."
- It is consumed once, in `internal/autoputer/run.go`, "before any actor runs", and passed into the runtime as `WithBootRestart`.

The boot (`Runtime.Start`, `internal/agentcore/runtime.go`):

- planned: `passivateInterruptedActivations` parks pre-boot runs for their one resume;
- crash: `closePreBootRunsAfterCrash` sets every pending, running, passivated or blocked run to cancelled with error `interrupted_by_restart`, and `closeOpenRoundsAfterCrash` gives every open round a system closure with that reason, which "wakes no one; a second boot finds no open round and writes nothing" (idempotent);
- both: engineering capsule reconcile, self-development closure after a crash, source schedules, then the projector. The comment that closes the function is the rule in code: "Boot does not directly deliver work ... the actor-wake outbox projector is the sole post-restart delivery path."

The gates on old work. Two gates stop a pre-boot wake from starting work after a crash, at the two places a wake could enter:

- the projector's `preBootWorkWake` marks a wake created before boot, of a kind in `RestartWorkKinds`, as interrupted;
- the actor handler's `preBootWorkOccurrence` refuses such an occurrence with `errInterruptedByRestart`, which wraps `actor.ErrDurableInvalid` so the dispatcher consumes it instead of retrying (`internal/actorruntime/handler.go`).

`RestartWorkKinds` lists the kinds that start or resume work: `initial_dispatch`, `coagent_result`, `channel_message`, `owner_revision`, `lifecycle_work_assigned`, the delegated-assignment and management resume deadlines, `source_poll_due`, and `inbox_record`. Cancels and fail-closed deadlines are absent on purpose: "they close work and still apply across a restart."

A bound on planned resumes. An apply that crashes the guest after writing its marker would read as planned on the next boot and apply again. `spendGuestApplyAttempt` keeps a durable count per reason and target and refuses after three starts (`guestApplyAttemptLimit`): "the first attempt, the planned resume that finishes it, and one replay." And `markPlannedRestart` clears the marker once the apply returns, because "a marker left behind would read a later crash as planned."

Standing intents. `recoverSourceSchedulesAtBoot` (`internal/agentcore/source_schedule.go`) implements the owner's 14 October refinement ("we need paper publishing 24/7, even after crashes"):

- planned boot: a schedule keeps its generation and its saved tick is armed again;
- crash boot: the poll in flight is recorded as interrupted, the crash counts against the schedule, the schedule gets a new generation (so the stale pre-boot wake is refused), and its next tick is armed one interval after boot, "never at once";
- a schedule whose tick was in flight at two crash boots in a row (`sourceScheduleCrashLoopLimit = 2`) is closed with that cause; an owner act restarts it;
- pending arrivals are closed "interrupted by a restart", never announced after the crash.

7.3 The incidents that wrote these rules

- **August: held-computer boot crash loops.** The owner's computer would not boot for weeks; startup reconcile hit a deterministic fatal path on every boot, and the problem was chased as four different root causes first (problem). This is the origin of "a crash never resumes".
- **12 October: a crash left work open and re-cast engineering.** Crash boots closed wakes but not the work items behind them, and an engineering reconcile re-cast restart-cancelled assignments up to three times (problem). The agreed fix was records-as-work plus round closures at boot. In today's code `closeOpenRoundsAfterCrash` exists, and the re-cast path (`engineering_desk.go`, `engineeringMaxRecastAttempts`) is gone from `internal/agentcore` (code: neither appears). The problem file still says "not fixed"; *inference:* it was repaired by deletion during Cut 1 and C1e and its status line was not updated.

- **13 October: crash boots re-drove updates.** Platform-update resume and self-development reconcile ran on every boot regardless of the marker (problem). The record carries an important warning: settling every crash-boot transition as "failed" would be false (the update may in fact be live), so crash settlement must read the updater's journal and record what it says. **"Close the work" does not mean "lie about the outcome."**
- **14 October: a planned refresh closed the paper's schedules.** A deploy refreshed the paper with a valid marker, but the marker carried work-state epoch 1 and the new binary reads 2, so the boot read as a crash, by design, and closed all 14 schedules; the paper was silent for about an hour until an owner act restarted them. vmctl logged `planned_restart=true` because its request succeeded; only the guest console showed the guest overruled it (problem). The same record found the paper had also been hibernated under memory pressure with no snapshot, and its wake was a markerless boot. Ruling 6 and `recoverSourceSchedulesAtBoot` came from this.

7.4 Where it is weak

- **An always-busy computer never takes a planned restart.** Deploys defer busy computers, and the paper is nearly always busy. The design note proposes a drain (stop starting queued tasks, let running ones finish within their deadline) but leaves who requests it as the owner's deploy-shape decision O6 (design §8).
- **Every non-planned stop is a crash.** Pressure hibernation without a snapshot, and every work-state epoch bump, read as crash boots. That is safe; it is also routine, so the closure path runs often and its visible fates must be cheap to act on.
- **The host cannot see the guest's verdict.** vmctl records the request, not the outcome. `/api/runtime/obligations` reports `restart.kind` for the latest boot (`internal/agentcore/obligations_surface.go`); nothing joins the two.
- **Process-local budgets reset per boot** (section 5.6).

7.5 Options

option	buys	cost
a drain before planned restarts	an always-busy computer can update	an owner decision about deploy shape (O6)
report the guest's boot kind back to vmctl	host logs tell the truth	small, red (vmctl)
snapshot hibernation as the default idle path	hibernate wakes resume memory, not boot	covered in 04-computer-lifecycle.md
epoch migrations that carry work forward	fewer crash boots at schema cuts	each cutover must prove its migration

8. One capture, end to end

A worked trace ties the pieces together. It is a composite assembled from the code paths above; the record ids are illustrative.

1. A source schedule's tick comes due. The wake kind is `source_poll_due`; the actor handler (`handleSourcePollDue`, `internal/actorruntime/handler.go`) polls the source "with no model call", records the poll's usage, arms the next tick, and the new captures arrive as a captures record from `source:<id>` in `management:<owner>`, with a wake row (`internal/agentcore/source_schedule.go`, `internal/store/inbox.go`).
2. The projector sees the signal, dispatches the wake into the actor log, and the dispatcher activates management's actor. `WakeRecordsInbox` finds no active management run and unread records, so it starts one.
3. Management's first cell calls `choir.Records()`, sees the captures record, and calls `choir.Delegate("intake", "", ids, "per_record")`. The tray holds one delegation intent; the cell ends.
4. The reducer commits, in one batch: a Cast from `management:<owner>` to `management-task:<id>` opening a round, the task row (queued), the lease (the open round), a wake for the task inbox, and management's cursor advance. Management ends its turn.

5. The task inbox's wake calls `startQueuedManagementTasks`. A slot is free, so the task's run starts in a fresh desk session.
6. The task's cell reads the capture, reads `choir.Stories()` (the host notes the library head it saw), and decides it is a new event: `choir.Cast("texture:new", headline, spec)`. It then calls `choir.Done("opened one story")`.
7. The reducer commits the open (the library head check passes), writes the disposition index for the capture, and the host mints the task's final Report: no residuals, so it is clean and does not wake management.
8. The new story's Texture inbox has a Cast; its wake starts a Texture run. The Texture cell reads the Cast, calls `choir.Write(edits, why)` and `choir.Done(...)`; the reducer commits one new revision through `ApplyTextureTurnWithSourceGraph`. Publication follows (05-publication-and-autopaper.md).

If the computer crashes between steps 5 and 7, the boot closes the task run and the task round as `interrupted_by_restart`, wakes no one, and the capture's fate is derived as "leased minus disposed". The schedule stays active and polls again one interval later. Management's next live turn may re-delegate the capture; that is new work by a live turn, not a resume.

Glossary

term	meaning
Actor	A unit with private state that handles one message at a time from its mailbox.
At-least-once	A delivery promise that may duplicate but never loses; made safe by idempotent handlers.
Cell	One model-written Go fragment, evaluated by Yaegi in a desk worker.
Cell-start snapshot	The records, document and pack shipped with a cell so it reads them without a round trip.
Commitment record	A typed, immutable, addressed act (Cast, Ask, Note, Reply, Report, Resolve, ...) on the ledger.
Conversation / round	The inbox pair talking; one request and its closing answer inside it.
Crash boot	A boot with no fresh, valid planned-restart marker. Pre-boot work is closed.
Cursor	A reader's durable "read up to here" mark, committed with its acts.
Dead-letter sink	Where a message goes after its retry budget is spent, so it stops looping.
Delegate	Management's verb to hand capture intake to management tasks (S2a).
Desk	A persistent root RLM agent: Texture, management, engineering or research.
Desk session / worker	The long-lived process holding one desk's Yaegi interpreter.
Fencing	Rejecting a commit from an activation older than the current epoch.
Head-of-line blocking	One slow item delaying everything behind it in a serial queue.
Idempotency key	A deterministic identity that makes a replayed write a no-op.
Lease	A bounded claim giving one worker the sole right to act on an item; in S2a, the open round.
Management task	A bounded sub-RLM of management with its own run, session and inbox.
Planned-restart marker	A durable file written before a deliberate restart and consumed once at boot.
Poisoned session	A Yaegi session whose code failed after it may have run; it must be respawned.
RLM	Recursive language model: a model loop whose code can start further model loops.
Reducer	The guest code (`commitTray`) that commits a successful cell's staged intents atomically.
Standing intent	An owner intent (a source schedule) that survives a crash; its occurrences do not.
Transactional outbox	Writing an outgoing message as a row in the same transaction as the state change.
Tray / staged intent	The per-cell buffer of acts a cell's verbs stage for the reducer.
Wake	A durable row that asks the dispatcher to activate a desk.
Work-state epoch	The schema version of durable work; a marker from another epoch reads as a crash.
Yaegi	A Go interpreter written in Go; Choir's cell evaluator.

Self-check

1. A management cell stages a Cast to a Texture and a Decline to a source, then panics on the next line. What reaches the store, what happens to the cursor, and can the desk's next cell reuse the session?
2. Why is the reader cursor a separate object rather than a field on the agent record, and what property would be lost if it were committed in a separate transaction from the desk's acts?
3. Trace the hops from a committed record to a started desk run. At which hops can a duplicate occur, and what stops each duplicate from doing harm?
4. In S2a, two intake tasks read the story library at the same head and both decide to open a story for the same event. Describe exactly what happens at commit, and why no lock was held during their model calls.
5. A deploy writes a valid planned-restart marker, the guest restarts on a new binary, and every source schedule closes. Give two independent reasons this can happen under today's code, and the rule that makes the closure safe rather than a bug.

Answer sketches

1. Nothing reaches the store: staged intents live in the tray, and a failed cell "reduces nothing and advances no cursor". The records stay unread and will be shown again. A panic is a runtime failure, so the session is poisoned (`unsafe_to_reuse`) and the next cell runs in a fresh worker with no earlier variables.
2. The agent record is rewritten under compare-and-set by many lifecycle transitions, so a cursor there would race unrelated writes. As its own object it commits in the same conditional batch as the acts. If the two committed separately, a crash between them would either mark records read whose answers were never committed (loss) or commit answers to records still unread (duplicates on replay).
3. Store commit (record + wake row, atomic) → projector sweep → actor log append → dispatcher activation → `WakeRecordsInbox`. Duplicates: the projector may dispatch before marking projected (actor log append is idempotent on UpdateID); the dispatcher retries an unincorporated update (fenced commit and serial-per-actor); `WakeRecordsInbox` sees an active run and does nothing, or sees no unread records and does nothing.
4. Each Cast to `texture:new` carries the library head its run saw. The first commit passes and advances the head; the second is stamped behind the current head and is refused under the commit lock, naming the story just opened, so that task feeds it in its next cell. This is optimistic concurrency: model calls take minutes, a commit takes milliseconds, and conflicts are rare.
5. The marker was refused because its work-state epoch differed from the new binary's (the 14 October cutover), or it was older than 15 minutes, unreadable, or not removable; or the restart was not the planned one (a pressure hibernate without snapshot, a real crash). Either way the boot is a crash boot. Today a crash boot keeps schedules active and closes only the poll in flight, so the paper keeps publishing; the closure that remains (two crash boots in a row with a poll in flight) is the crash-loop guard.

Read these three files

1. **`internal/yaegikernel/choir.go`.** Read `deskModuleSets`, `ChoirExports` and `NewChoirScope` together: the first is the authority table, the second is how it becomes a namespace, the third is the handle that sits behind it (and is broader than the namespace). Then read `Delegate` for a verb that validates its arguments against the cell-start snapshot.
2. **`internal/agentcore/rlm_reduce.go`.** Read `commitTray`, `intentIdempotencyKey` and `recoverPartialActCommit`. They are the clearest example in the codebase of making a multi-store write safe by deterministic identity and replay.
3. **`internal/agentcore/planned_restart.go`.** Short and complete: the marker's crash-safe write, its once-only consumption, the epoch and age checks, `RestartWorkKinds`, and the apply attempt budget. Every rule in section 7 is visible here in under 450 lines.

Sources and confidence

- **Code-read** (at 980ffaf4e): `internal/yaegikernel/{choir.go, session_loop.go, desk_session_worker.go, desk_worker_spawn_linux.go, desk_worker_memlimit_linux.go, allowlist.go, intent.go, eval.go, session.go}`; `internal/agentcore/{tools_desk.go, rlm_reduce.go, inbox_records.go, actor_wake_projector.go, runtime.go, planned_restart.go, source_schedule.go, management_tasks.go, research_children.go, engineering_assignment_runtime.go, engineering_assignment_fate.go}`; `internal/store/{inbox.go, actor_wake_signal.go, texture_turn.go, management_tasks.go}`; `internal/actor/{actor.go, dispatcher.go, log_sqlite.go}`; `internal/actorruntime/{adapter.go, handler.go}`; `internal/agentprofile/agentprofile.go`; `internal/autoputer/run.go`; `internal/textureowner/texture_cell_author.go`; `internal/workitem/fingerprint.go`; `internal/runtimeprompts/overlays/rlm_management_runtime.yaml`; `internal/provideriface/config.go`.
- **Doc** (numbers quoted, not re-measured): CPU efficiency, management delegation design and adjudication, LR lever 6, guest memory return, state and vision §3.3 and §4.4, isolation review, doctrine (I1, I3, I5, I18, I19, I21, I26), `CLAUDE.md` ("Restarts End Work"), and the problem records linked in each section.
- **Inference** (marked in the text): that a session handle broader than the desk's verbs weakens the broker as a second layer; that idle workers persist on a quiet computer because reclaim is lazy; that the delivery path's two logs complicate diagnosis; that the 12 October problem was repaired by deletion without a status update; the composite trace in section 8.
- **Not covered here:** the Texture controller's own wake logic (`internal/textureowner/texture_controller.go`), capsule internals (see the isolation review), self-development operations beyond their restart behaviour, and the model calls themselves (03-gateway-and-providers.md).